

# XGBoost: Extreme Gradient Boosting

## Modello

### Logica dell'Algoritmo

**XGBoost** (Extreme Gradient Boosting) è un **ensemble method sequenziale** che costruisce alberi decisionali uno dopo l'altro, dove ogni nuovo albero **corregge gli errori del precedente**.

A differenza di **Random Forest** (alberi paralleli indipendenti), XGBoost è un **boosting algorithm** che combina predizioni tramite:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Dove:

- $\hat{y}_i^{(t-1)}$  è la previsione cumulativa dopo t-1 alberi
- $f_t(x_i)$  è il nuovo albero che predice i residui dell'iterazione precedente

### Obiettivo Regolarizzato

XGBoost minimizza un obiettivo di loss regolarizzato:

$$L(\phi) = \sum_{i=1}^n l(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k)$$

Dove:

- Primo termine: **loss function** (errore tra previsione e realtà)
- Secondo termine: **regolarizzazione** che penalizza la complessità dell'insieme di alberi

La regolarizzazione è:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda ||w||^2$$

Dove:

- $\gamma T$  penalizza il numero di foglie (T)
- $\frac{1}{2} \lambda ||w||^2$  penalizza la magnitudine dei pesi delle foglie

**Interpretazione:** il termine di regolarizzazione preferisce alberi semplici e pesi piccoli, **riducendo overfitting**.

### Gradient Tree Boosting (Secondo Ordine)

Aniché minimizzare direttamente, XGBoost usa un **secondo-ordine approximation** della loss:

$$L^{(t)} \approx \sum_{i=1}^n \left[ g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Dove:

- $g_i = \frac{\partial l}{\partial \hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$  è il **primo-ordine gradiente** (come in GBM standard)
- $h_i = \frac{\partial^2}{\partial \hat{y}^{(t-1)^2}} l(y_i, \hat{y}^{(t-1)})$  è il **secondo-ordine gradiente** (novità di XGBoost)

L'uso del secondo-ordine consente un'**ottimizzazione più accurata** rispetto a metodi che usano solo il primo-ordine.

### Scoring Function per Split

Per una struttura d'albero fissa, il **quality score** è:

$$\text{Score}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

Dove:

- $I_j$  è l'insieme di istanze nella foglia  $j$
- Questo score misura quanto una struttura d'albero è "buona"
- Valori negativi più grandi (più negativi) = alberi migliori

## Complessità Computazionale

### Training (Esatto)

$$O(K \cdot d \cdot n \log(n))$$

Dove:

- $K$  = numero di alberi (iterazioni)
- $d$  = profondità massima dell'albero
- $n$  = numero di osservazioni
- $\log(n)$  = sorting dei dati per trovare gli split

Per ogni albero, l'algoritmo "exact greedy":

- Ordina i dati per feature:  $O(n \log n)$  once per feature
- Enumera split con accumulo di gradienti:  $O(n)$

### Training (Approssimato - con Quantile Sketch)

$$O(K \cdot d \cdot n)$$

Usando il **weighted quantile sketch** (novità di XGBoost):

- Propone candidate split points in modo intelligente
- Non richiede sorting completo
- Ottimale per dataset out-of-core

### Inference (Previsione)

$$O(K \cdot d)$$

Dove  $K$  è il numero di alberi,  $d$  la profondità media:

- Attraversare  $K$  alberi sequenzialmente
- Per ogni albero, seguire il cammino dalla radice alla foglia (profondità  $d$ )

### Memoria

$$O(K \cdot n \cdot d + \text{block structure})$$

- Memorizzare  $K$  alberi:  $O(K \cdot n)$  nel peggiore (tutte le istanze in ogni albero)
- Block structure per parallelizzazione:  $O(n)$  aggiuntivo

### Scalabilità: Conclusioni

#### Vantaggi rispetto a Random Forest:

- **Sequenziale è più efficiente:** ogni albero è costruito per correggere errori specifici
- **Alberi più piccoli:** con boosting, alberi più shallow (tipicamente profondità 5-8) vs RF (profondità  $\leq n$ )

- **Meno alberi necessari:** 100-500 alberi vs RF 1000+
- **Out-of-core:** XGBoost supporta nativamente dati che non fit in memoria

#### Svantaggi:

- **Sequenziale:** training è sequenziale, non parallelizzabile come RF
- **Sensibile ai parametri:** il tuning di learning rate, depth, regularization è critico

**Trade-off:** XGBoost è ~10x più veloce di Random Forest per item su dataset moderati (Chen & Guestrin, 2016).

## Rappresentazione Interna

### Struttura del Modello

XGBoost rappresenta internamente il modello come:

Ensemble = [Tree\_1, Tree\_2, ..., Tree\_K]

Previsione =  $\sum_k f_k(x)$

Dove ogni Tree\_k è:

- Struttura: split feature, threshold, default direction
- Pesì: predizioni per ogni foglia
- Gradienti: accumulati durante training

### Differenze da Random Forest

| Aspetto            | Random Forest                | XGBoost                           |
|--------------------|------------------------------|-----------------------------------|
| <b>Alberi</b>      | Indipendenti, paralleli      | Sequenziali, dipendenti           |
| <b>Costruzione</b> | Campioni casuali (bootstrap) | Su residui dell'albero precedente |
| <b>Profondità</b>  | Spesso profonde (10-20)      | Solitamente basse (3-8)           |
| <b>Numero</b>      | Tanti (500-2000)             | Meno (50-500)                     |
| <b>Pesi</b>        | Voting majority              | Somma pesata                      |

### Implicazioni per la Spiegabilità

#### Contro:

- Ancora più opaca di RF (500 alberi è molto)
- L'ordine degli alberi importa (sequenziale vs parallelo)
- Difficile tracciare quale albero ha causato una previsione

#### Pro:

- Feature importance è più **stabile** (alberi correlati meno che in RF)
- SHAP (sviluppato dagli stessi autori) è ottimizzato per XGBoost
- Ogni albero corrisponde a una "correzione" specifica di errori

## Vincoli sui Dati

### Nessuna Assunzione Strutturale

Come Random Forest e Decision Trees, XGBoost **non assume**:

- Linearità
- Normalità
- Omoschedasticità

## Bilanciamento delle Classi

XGBoost è **più robusto** di DT singolo grazie alla regolarizzazione, ma può comunque soffrire di classi sbilanciate:

### Soluzioni:

- **Scale\_pos\_weight**: parametro che pesa la classe minoritaria
- **Stratified k-fold**: durante cross-validation, mantenere proporzioni
- **Threshold tuning**: non usare 0.5, usare soglia ottimale

## Normalizzazione delle Feature

A differenza di SVM, XGBoost **non richiede** normalizzazione:

- Usa split basati su percentili (invariante a scala)
- Gestisce naturalmente feature con range diversi

## Gestione di Dati Sparsi

### Novità importante di XGBoost: algoritmo sparsity-aware

Quando una feature ha valore mancante:

- L'istanza viene instradata ad una **default direction** (sinistra o destra)
- La default direction è **imparata dai dati**, non fissa
- Computazione è **lineare nel numero di non-missing entries** (50x più veloce su dati sparsi)

## Dati Ponderati

XGBoost supporta **instance weights** tramite **weighted quantile sketch**:

- Calcola percentili ponderati (non solo uniforme)
- Teoricamente garantito a mantenere accuracy nei candidate splits

## Capacità Predittive

### Punti di Forza

#### 1. Eccellente Performance Generale

- Ha vinto 17 di 29 Kaggle competitions nel 2015
- Spesso outperform altri metodi anche ensemble
- Performance quasi universalmente buona su dati reali

#### 2. Flessibile per Problemi Diversi

- Classificazione binaria e multi-classe
- Regressione
- Learning to Rank
- Ranking object detection
- Supporta custom loss functions

#### 3. Robusto a Dati Imperfetti

- Gestisce missing values
- Dati sparsi (one-hot encoding, etc.)
- Classe sbilanciate (con tuning)
- Outlier (soft-margin mitiga)

#### 4. Pattern Non-Lineari e Interazioni

- Cattura relazioni complesse

- Interazioni automatiche tra feature
- Nessun feature engineering necessario (spesso)

## 5. Regolarizzazione Incorporated

- Riduce overfitting intrinsecamente
- Non serve validation set separato per early stopping
- Generalizza bene con pochi dati

## Punti di Debolezza

### 1. Tuning Complesso

- Molti hyperparameter (learning rate, depth, lambda, gamma, etc.)
- Sensibile al tuning
- Default non sempre ottimali per il problema

### 2. Interpretabilità Scarsa

- 100-500 alberi è difficile da ispezionare
- Feature importance non è univoca (molti metodi differenti)

### 3. Non Probabilistico Nativo

- Output è predizione, non probabilità calibrate
- Per probabilità, usare sigmoid transformation

### 4. Sequenziale = Lento a Trainare

- Alberi dipendono dal precedente (non parallelizzabili)
- Più lento di RF puro per training

### 5. Curva di Apprendimento Steep

- Molti parametri da capire
- Documentazione densa
- Non intuitivo come Decision Tree o LR

## Metriche per la Confidenza

### Metriche Standard di Classificazione

Stesse metriche di altri classificatori:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:**  $(TP + TN) / (TP + TN + FP + FN)$
- **Precision, Recall, F1-Score:** standard
- **ROC Curve e AUC:** trade-off tra TPR e FPR

### Margin/Distance Score

XGBoost produce un **margin score** (somma dei contributi degli alberi):

$$\text{margin}_i = \sum_{k=1}^K f_k(x_i)$$

### Interpretazione:

- Valori più alti = più confidenza nella classe positiva
- Valori più bassi = più confidenza nella classe negativa
- Valori vicini a 0 = incertezza

## Probabilità Calibrate (Sigmoid Transform)

Per ottenere probabilità da margin score:

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(-\text{margin})}$$

## Early Stopping e Monitoring

XGBoost supporta **early stopping** usando metriche di validazione:

- Monitor AUC, log-loss, RMSE su validation set
- Stop quando metrica non migliora per N iterazioni
- Previene overfitting automaticamente

**Vantaggio:** non serve splitting train/val/test esplicito (uses internal validation).

## Metriche per la Comprensione e Spiegabilità

### 1. Feature Importance (Gain-Based)

Misura la **riduzione di loss media** dovuta a ogni feature:

$$\text{Importance}_j = \frac{1}{K} \sum_{k=1}^K \text{Gain}_j^{(k)}$$

Dove  $\text{Gain}_j^{(k)}$  è la riduzione di loss totale quando feature j fa split nell'albero k.

**Interpretazione:**

- Feature con gain alto sono "importanti" per il modello
- Somma a 100% (normalizzato)

**Caveat:** misure gain possono essere biased verso feature ad alta cardinality.

### 2. Feature Importance (Split-Based)

Conta quante volte una feature è stata usata per split:

$$\text{Cover}_j = \frac{1}{K} \sum_{k=1}^K \# \text{ splits su feature } j$$

**Interpretazione:** feature più frequenti sono considerate "importanti".

### 3. Feature Importance (SHAP)

**SHAP** (SHapley Additive exPlanations) è stato sviluppato specificamente per XGBoost:

- Assegna ogni predizione tra le feature usando valori Shapley
- Teoricamente fondata (teoria dei giochi)
- Fornisce spiegazioni sia globali che locali
- XGBoost ha ottimizzazioni native per SHAP

**Utilizzo:**

```
import shap
explainer = shap.TreeExplainer(xgboost_model)
shap_values = explainer.shap_values(data)
shap.summary_plot(shap_values, data)
```

### 4. Partial Dependence Plot (PDP)

Mostra come la previsione media cambia al variare di una feature:

$$\text{PDP}_j(x_j) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_j, x_{-j}^{(i)})$$

**Visualizzazione:** linea/curva che mostra relazione feature-output.

## 5. Individual Conditional Expectation (ICE)

Versione per-istanza di PDP:

- Linee separate per ogni campione
- Capire se l'effetto è uniforme o varia

## 6. Tree Visualization

Per problemi specifici, è possibile visualizzare alberi individuali:

```
import xgboost as xgb
xgb.plot_tree(model, num_trees=0) # Plot primo albero
```

**Limite:** con 500 alberi, visualizzare anche uno solo è complesso.

## 7. Explain Single Prediction (Contrastive)

Per una singola istanza, estrarre i contributi:

```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_instance)
shap.force_plot(explainer.expected_value, shap_values, X_instance)
```

Questo mostra:

- Valore base (expected value)
- Contributi positivi (push verso +)
- Contributi negativi (push verso -)

## Limiti di Predizione

### Sensibilità al Tuning di Iperparametri

XGBoost ha **molte iperparametri critici**:

1. **learning\_rate** (eta): shrinkage dopo ogni albero
  - Piccolo (0.01-0.1) = più stabile, training più lungo
  - Grande (0.3-1.0) = convergenza veloce, rischio overfitting
2. **max\_depth**: profondità massima alberi
  - 3-5: shallow, underfitting
  - 10-15: typical, balanced
  - 20+: deep, overfitting
3. **min\_child\_weight**: istanze minime in foglia
  - Alto = underfitting, modello semplice
  - Basso = overfitting, modello complesso
4. **gamma**: perdita minima per split
  - Alto = meno split (underfitting)
  - Basso = più split (overfitting)
5. **lambda, alpha**: regolarizzazione L2, L1

- Alto = coefficienti piccoli (underfitting)
- Basso = coefficienti grandi (overfitting)

**Problema:** combinazioni non-lineari. Tuning è un'arte, non scienza.

### Difficoltà su Dati Lineari

Se la vera relazione è lineare e semplice:

- LR è più efficiente (pochi parametri)
- XGBoost "forza" una soluzione tree-based (molti alberi per una retta)

### Scarsa Generalizzazione con Molto Rumore

Se il dataset ha very high noise-to-signal ratio:

- XGBoost potrebbe overfitting nonostante regolarizzazione
- Mitigare con higher lambda, lower learning rate

### Extrapolazione Debole

Come Random Forest, XGBoost **non** **extrapola**:

- Predizioni rimangono nel range osservato
- Non adatto per forecast trends

### Limiti di Spiegabilità

#### Opacità della Struttura Sequenziale

Il fatto che ogni albero dipende dal precedente rende:

- Non chiaro quale albero "causa" una previsione
- Difficile tracciare il ragionamento passo per passo
- Un albero da solo potrebbe dare previsione completamente diversa

#### Interpretazione di Feature Importance Ambigua

XGBoost ha **tre metodi diversi** per feature importance (Gain, Cover, Frequency):

- Possono dare ranking **completamente diversi**
- Quale è "giusto"? Dipende dal caso d'uso
- Nessuno è intrinsecamente "corretto"

### Difficoltà di Comunicazione

Spiegare una previsione di XGBoost a non-esperti è difficile:

- "Il modello è fatto di 200 alberi che si sommano"
- Non è intuitiva come "se Feature A > X allora Y" (DT)
- Non è semplice come "il coefficiente di A è 0.5" (LR)

### SHAP è Computazionalmente Costoso

SHAP è il metodo migliore per spiegare XGBoost, ma:

- Calcolo è  $O(K \cdot 2^p)$  nel peggiore (exponential in # features)
- Per 100+ features, diventa prohibitively slow
- TreeSHAP (ottimizzato) è più veloce ma comunque caro

### Instabilità di Feature Importance

Se parametri cambiano leggermente:

- Feature importance può cambiare drasticamente



- Spiegazioni non sono "robuste"

## Confronto con altri Algoritmi

### Vs. Random Forest

| Aspetto                   | Random Forest                | XGBoost                     |
|---------------------------|------------------------------|-----------------------------|
| <b>Velocità Training</b>  | Veloce (parallelo)           | Lento (sequenziale)         |
| <b>Velocità Inference</b> | Veloce                       | Veloce                      |
| <b>Performance</b>        | ★★★★ Buona                   | ★★★★★ Eccellente            |
| <b>Overfitting</b>        | ★★ Basso                     | ★ Basso ma sensibile tuning |
| <b>Interpretabilità</b>   | ★★ Media                     | ★★ Media (ma con SHAP)      |
| <b>Tuning</b>             | ★★★★ Semplice                | ★ Complesso                 |
| <b>Quando usare</b>       | Velocità + buona performance | Massima performance         |

### Vs. Logistic Regression

| Aspetto                    | LogR                        | XGBoost                   |
|----------------------------|-----------------------------|---------------------------|
| <b>Interpretabilità</b>    | ★★★★★ Alta                  | ★★ Bassa                  |
| <b>Pattern Non-Lineare</b> | ★ Scarso                    | ★★★★★ Eccellente          |
| <b>Tuning</b>              | ★★★★ Semplice               | ★ Complesso               |
| <b>Scalabilità</b>         | ★★★★★ Buona                 | ★★★★ Media                |
| <b>Dati Rari</b>           | ★★★★★ Buono                 | ★★★★ Buono (needs tuning) |
| <b>Quando usare</b>        | Interpretabilità + baseline | Performance è prioritario |

### Vs. Neural Networks

| Aspetto                 | Neural Networks           | XGBoost                       |
|-------------------------|---------------------------|-------------------------------|
| <b>Capacità</b>         | ★★★★★ Illimitata          | ★★★★★ Molto buona             |
| <b>Dati Necessari</b>   | ★ Molti (milioni)         | ★★★★★ Meno (migliaia)         |
| <b>Training</b>         | ★★ Lungo                  | ★★★★ Veloce                   |
| <b>Interpretabilità</b> | ★ Nessuna                 | ★★ SHAP disponibile           |
| <b>Quando usare</b>     | Big data + pattern visivo | Structured data + performance |

### Vs. LightGBM / CatBoost

**LightGBM** (Light Gradient Boosting Machine):

- Simile a XGBoost ma con **histogram-based split finding**
- ~2x più veloce di XGBoost
- Più basso memoria
- Meno maturo (comunità più piccola)

**CatBoost** (Categorical Boosting):

- Specializzato in dati categorici
- Gestisce nativamente categorie senza encoding
- Meno tuning necessario (default migliori)

- Performance leggermente superiore a XGBoost su dati categorici

**Confronto:**

| Aspetto                | XGBoost | LightGBM | CatBoost |
|------------------------|---------|----------|----------|
| <b>Velocità</b>        | ★★★★★   | ★★★★★★   | ★★★★★    |
| <b>Performance</b>     | ★★★★★★  | ★★★★★★   | ★★★★★★   |
| <b>Tuning Facile</b>   | ★★★     | ★★★★     | ★★★★★    |
| <b>Dati Categorici</b> | ★★★     | ★★★★     | ★★★★★★   |
| <b>Comunità</b>        | ★★★★★★  | ★★★★★    | ★★★★     |
| <b>Stabilità</b>       | ★★★★★★  | ★★★★★    | ★★★★     |

## Varianti e Estensioni

### 1. XGBoost Rank (Learning to Rank)

Specializzato per problemi di ranking:

- Ottimizza NDCG, MAP, ecc.
- Usato in search engines, ad ranking
- Gestisce query groups

### 2. XGBoost GPU

Supporto nativo per GPU NVIDIA (CUDA):

- ~10x più veloce di CPU
- Per dataset molto grandi
- Richiede GPU con memoria sufficiente

### 3. XGBoost per Distributed Training

- **Spark integration:** XGBoost su Spark cluster
- **Hadoop/MPI:** per distributed environments
- **Dask:** per parallel computing su CPU

### 4. Dart (Dropouts Meet Multiple Additive Regression Trees)

Variante di XGBoost che usa **dropout** (come neural networks):

- Dropa (scarta) alberi randomicamente durante training
- Riduce correlazione tra alberi
- Spesso migliore performance, specialmente su dataset piccoli

### 5. Linear Booster

Anziché tree boosting, usa **regressione lineare** a ogni step:

- Più semplice e interpretabile
- Più veloce
- Per pattern principalmente lineari

## Raccomandazioni Pratiche

### Quando Usare XGBoost

✓ **Ottime scelte:**

- Competizioni di ML (Kaggle, ecc.)
- Structured tabular data (no immagini/testo)
- Necessità di performance massima
- Dataset da migliaia a milioni di righe
- Mix di feature categoriche e numeriche
- Dati sparsi (one-hot encoding, ecc.)

### ✗ Cattive scelte:

- Testo/immagini (usare Deep Learning)
- Massima interpretabilità richiesta (usare LR o DT)
- Real-time inference su edge devices (slow inference)
- Dataset piccolissimi (< 1000 righe, rischio overfitting)
- Dati molto rumorosi (difficile tunare)

### Workflow Consigliato

1. BASELINE: Train LR o DT semplice  
↓
2. PARAMETER TUNING: Grid search / Random search / Bayesian
  - Iniziare con default: learning\_rate=0.1, max\_depth=6, lambda=1
  - Tuning in ordine: learning\_rate → max\_depth → lambda/alpha
 ↓
3. VALIDATION: Usare 5-10 fold cross-validation  
↓
4. MONITORING: Watch AUC/RMSE su validation set
  - Early stopping quando validation metric plateaus
 ↓
5. INTERPRETATION: Usare SHAP per understand feature importance

### Hyperparameter Tuning Dettagliato

#### Step 1: Learning Rate e Number of Rounds

```
# First: find good number of trees with default learning rate
xgb_params = {
    'eta': 0.1,
    'max_depth': 6,
    'min_child_weight': 1,
    'gamma': 0,
    'lambda': 1,
    'alpha': 0
}
# Train con early stopping, find best num_rounds
```

#### Step 2: Tree Depth e Min Child Weight

```
# Grid search over depth e min_child_weight
depths = [3, 5, 7, 9]
min_child_weights = [1, 3, 5, 7]
# Risultato: profondità tipicamente 5-8, min_child 1-3
```

#### Step 3: Regularizzazione (Lambda, Alpha)

```
# Una volta trovati depth e learning_rate
# Tuare lambda (L2) e alpha (L1)
lambdas = [0.1, 1, 10, 100]
alphas = [0, 0.1, 1, 10]
```

#### Step 4: Column Subsampling

```
# Ridurre numero di feature per split
# Aiuta con overfitting
colsample_bytree = [0.5, 0.7, 1.0]
colsample_bylevel = [0.5, 0.7, 1.0]
```

#### Feature Engineering Minimo Necessario

XGBoost è **robusto a feature engineering scadente**, ma alcuni trucchi aiutano:

1. **One-hot Encoding** per categoriche (XGBoost le gestisce, ma sparsity-aware accelera)
2. **Log Transform** per feature long-tailed (es. income)
3. **Interaction Features** (a volte aiuta):  $x_1 \times x_2$
4. **Polynomial Features** (raramente necessario)

**In generale:** meno feature engineering, meglio è. Lasciar decidere a XGBoost.

#### Debugging Overfitting

Se il modello overfits (validation error cresce):

- Aumentare `lambda` (L2 regolarizzazione)
- Diminuire `learning_rate`
- Ridurre `max_depth`
- Aumentare `min_child_weight`
- Aggiungere column/row subsampling
- Aumentare `gamma`
- Usare early stopping più aggressivo

#### Debugging Underfitting

Se il modello underfits (training error alto):

- Diminuire `lambda`
- Aumentare `learning_rate`
- Aumentare `max_depth`
- Diminuire `min_child_weight`
- Aumentare numero di rounds
- Disabilitare early stopping (train longer)

#### Considerazioni Etiche e Normative

##### GDPR Right to Explanation

XGBoost è un **black-box model** in contesti sensibili (credito, assicurazione):

- Necessaria spiegabilità per ogni decisione
- Usare SHAP per soddisfare requisiti GDPR
- Documentare feature importance

##### Fairness e Bias

Controllare per bias rispetto a protected attributes:

- Gender, Race, Age, etc.
- Usare fairness metrics (disparate impact ratio, etc.)
- Monitor SHAP values per protected groups

#### Prompt